Operating System

Assignment- 7

Question 1 — Implementing User-Level Threads

Topic: Threads

xv6 does not natively support multithreading within a single process (it only supports fork(), which creates a new process with its own address space). In this question, you will implement basic kernel-supported threads that share the same address space.

Tasks:

a) Implement a new system call int clone(void(*fcn)(void*), void *arg, void *stack) that creates a new thread sharing the same address space (pgdir) as the calling process, but with its own kernel stack, and begins executing fcn(arg) on the user-provided stack.

b) Implement a corresponding int join(void) system call that waits for a thread created by clone() to finish and cleans up its resources.

c) Modify struct proc as needed (e.g., to mark a process as a thread and identify its shared address space/parent).

d) Ensure exit(), fork(), and wait() correctly handle thread processes (e.g., a thread should not have its own copy of the address space freed on exit, since it's shared).

e) Write a user program threadtest.c that creates 3 threads using clone(), each incrementing a shared global counter 100,000 times, then calls join() on each. Print the final counter value.

Expected Outcome: All threads execute in the same address space (verify by observing the shared counter is visible across threads). Without synchronization, the final value will likely be less than 300,000 due to race conditions — this sets up Question 2.

Question 2 — Process/Thread Synchronization using Semaphores

Topic: Synchronization

Using the threads implemented in Question 1 (or plain fork()-based shared memory if Question 1 is not attempted), implement counting semaphores in xv6 to fix the race condition observed above.

Tasks:

a) Implement a semaphore structure struct semaphore { int value; struct spinlock lk; ... } in the kernel, using xv6's existing sleep()/wakeup() mechanism for blocking/waking processes.

b) Implement the following system calls:

System Call	Description
int sem_init(int sem_id, int value)	Initializes semaphore sem_id with the given value
int sem_wait(int sem_id)	Decrements the semaphore; blocks if value < 0
int sem_signal(int sem_id)	Increments the semaphore; wakes a waiting process if any

c) Maintain a fixed-size array of semaphores (e.g., 10) in kernel space, protected by a spinlock.

d) Modify threadtest.c from Question 1 into synctest.c: wrap the shared counter increment in sem_wait()/sem_signal() calls around a critical section, and verify the final counter value is now exactly correct (e.g., 300,000).

e) Additionally, implement the classic Producer-Consumer problem using your semaphores: one producer thread/process inserts items into a fixed-size buffer (size 5), one consumer removes them, using semaphores for empty, full, and mutex.

Expected Outcome: The shared counter is exact with synchronization (compare against Q1's incorrect value). Producer-Consumer runs without buffer overflow/underflow, verified by printed log of produced/consumed items.

Question 3 — Deadlock Detection

Topic: Deadlock

Simulate a resource-allocation system in user space and implement the Banker's Algorithm / Deadlock Detection Algorithm to detect unsafe states, using xv6 only as the execution environment (this is primarily a user-program question, since xv6 itself has no multi-resource allocator).

Tasks:

a) Write a user-space program deadlock.c in xv6 that simulates n processes and m resource types using static arrays: Allocation[n][m] (resources currently allocated to each process), Request[n][m] or Max[n][m] (resources each process may still request), and Available[m] (currently available resources of each type).

b) Implement a function int detect_deadlock() that applies the deadlock detection algorithm: find a process whose Request can be satisfied by Available, simulate it finishing and add its allocation back to Available, repeat — any process left unfinished indicates a deadlock. Return the list of deadlocked processes (if any).

c) Simulate an actual deadlock scenario using real xv6 processes and pipes (or shared memory via clone() from Q1) — e.g., Process A holds Pipe1's write end and waits to read Pipe2; Process B holds Pipe2's write end and waits to read Pipe1 — and show that your detection function correctly identifies the circular wait.

d) Extend the program to also implement deadlock avoidance using the Banker's Algorithm: given a Max[n][m] matrix, implement int is_safe_state() that checks if a hypothetical resource request can be safely granted without leading to deadlock.

Expected Outcome: The program correctly identifies deadlock/no-deadlock for at least 3 different test matrices (one deadlocked, one safe, one that becomes unsafe after a request), with printed state (Available, Allocation, Need) at each step.

Question 4 — Priority-Based Preemptive CPU Scheduling

Topic: Scheduling

Replace xv6's default Round-Robin scheduler with a priority-based preemptive scheduler with aging.

Tasks:

a) Add an integer priority field (1 = highest, 20 = lowest) to struct proc; default priority on creation = 10.

b) Implement system calls int set_priority(int pid, int priority) and int get_priority(int pid).

c) Modify scheduler() in proc.c to always select the RUNNABLE process with the lowest priority number; break ties in round-robin order.

d) Implement aging: maintain a wait-tick counter per process; if a RUNNABLE process waits more than 30 ticks without being scheduled, increment its priority (decrement the number) by 1 and reset the counter.

e) Write priotest.c: fork 4 processes with distinct priorities (including one very low priority, e.g., 20), each running a CPU-bound loop; print completion order and confirm that higher-priority processes generally finish earlier, and that the lowest-priority process does not starve (it eventually completes, thanks to aging).

Expected Outcome: Completion order correlates with priority, and the aging mechanism is demonstrably preventing starvation (shown via printed wait-tick/priority changes in the lowest-priority process over time).

Common Submission Requirements (for all 4 questions)

a) All modified kernel source files (proc.c, proc.h, sysproc.c, syscall.c, syscall.h, defs.h as needed), clearly commented.

b) User test programs: threadtest.c, synctest.c, deadlock.c, priotest.c.

c) Terminal output screenshots for each question demonstrating correct behavior.

d) A consolidated report (2–3 pages) covering: design decisions and key data structures for each part; problems faced and how they were resolved; for Q1–Q2, a before/after comparison of the race condition; for Q3, deadlock detection results across all test cases; for Q4, a scheduling behavior graph/table (completion time vs. priority).